

AFTER THE COURSE

Next steps

Exam preparation, learning path and features beyond the exam

Contents

PART 1	Before the exam	3
PART 2	Learning path after the Associate	4
PART 3	Metadata-driven pipelines	5
PART 4	Lakeflow Connect beyond SQL Server	6
PART 5	System tables and observability	7
PART 6	OpenSharing (formerly Delta Sharing)	8
PART 7	Lineage	8
PART 8	AI/BI dashboards and Genie	9
PART 9	Terraform for the environment	9
PART 10	CI/CD with GitHub Actions and bundles	10

This three-day course ends with you prepared for the Databricks Certified Data Engineer Associate exam. This guide covers what comes next: how to book and prepare for the exam, which credentials and courses follow the Associate level, and which platform features are worth learning even though the exam does not test them. Most sections point to material in this course repository that you can reuse.

PART 1

Before the exam

Booking

Step	Detail
Review the exam	databricks.com/learn/certification : exam page, current exam guide and FAQ
Create an account	webassessor.com/databricks , the registration portal run by Kryterion
Register	<i>Register for an Assessment</i> , choose Data Engineer Associate, then online proctored or a test centre
Prepare the setup	For online delivery, run the system check on the computer you will use, with a webcam and a quiet room; have valid photo ID ready
After passing	The badge and certificate appear at credentials.databricks.com ; they are valid for two years

The fee is USD 200 plus applicable taxes. Check the retake and rescheduling rules in the certification FAQ before you book, because they can change. Book a date two to four weeks after the course, while the material is still fresh.

Preparation approach

1. Work through the *Exam objectives map* from this course, one section per session. It lists the notebooks to reopen and the facts to know for every objective.
2. Redo the labs without the guides, from an empty cell. Rebuilding the code from memory prepares you far better than rereading solutions.
3. Take the course quizzes again, and a practice exam under timed conditions. Aim for at least 80% before the real exam.
4. For every wrong answer, find the matching objective and write down why the correct option wins. The exam uses scenarios, so the reasoning matters more than memorising syntax.
5. Download the current exam guide two weeks before the exam and compare it with the map. Databricks revises the guide every few months.

Recommended Databricks Academy courses

The exam guide recommends the instructor-led course *Data Engineering with Databricks* and the following self-paced courses in Databricks Academy. Use them to fill specific gaps rather than as a complete second course.

Self-paced course	Exam sections it supports
Data Ingestion with Lakeflow Connect	2 Data Ingestion and Loading
Build Data Pipelines with Lakeflow Spark Declarative Pipelines	2, 3 Data Transformation and Modeling
Deploy Workloads with Lakeflow Jobs	4 Working with Lakeflow Jobs, 6 Troubleshooting
DevOps Essentials for Data Engineering	5 Implementing CI/CD
Data Interoperability with Unity Catalog	1 Platform, 7 Governance and Security
Get Started with Data Governance on Databricks	7 Governance and Security

Databricks Academy is at databricks.com/learn/training. Partner and customer accounts may have access to additional content.

PART 2

Learning path after the Associate

Certifications

Credential	Focus	When it makes sense
Data Engineer Professional	Advanced pipelines, performance, security, testing and deployment at scale	After about a year of production data engineering on Databricks
Associate Developer for Apache Spark	DataFrame API and Spark SQL, independent of the platform	If you also work with Spark outside Databricks
Data Analyst Associate	Databricks SQL, dashboards, analytics	If you serve BI teams or build dashboards
Machine Learning Associate and Professional	Feature engineering, MLflow, model deployment	If you move towards ML engineering
Generative AI Engineer Associate	Retrieval-augmented generation, agents, model serving	If you build LLM applications on Databricks
Platform Administrator and Platform Architect accreditations	Workspace administration, cloud architecture on Azure, AWS or GCP	If you own the platform setup

The full list, including newer credentials, is on databricks.com/learn/certification. Before choosing a next exam, compare its current guide with your daily work.

Practising without a company workspace

Databricks Free Edition replaces the former Community Edition. It gives you a serverless-only workspace with default storage, Unity Catalog, notebooks, jobs, pipelines, SQL and AI/BI. Usage is limited by per-account quotas, outbound internet access is restricted to trusted domains, and commercial use is not allowed. It is well suited to repeating the labs from this course. Sign up at databricks.com/learn/free-edition.

Other options: a 14-day trial workspace on Azure, AWS or GCP, or a workspace built from `infra/terraform` in this repository (Part 9).

PART 3

Metadata-driven pipelines

In the course, each dataset had its own pipeline code. Real projects often ingest dozens or hundreds of similar sources. A **metadata-driven** pipeline reads a configuration table when an update starts and generates the datasets in a loop. Onboarding a new source then means inserting one row, not writing new code.

The folder `materials/lakeflow/lakeflow_metadata_driven` contains a working example. `00_create_config_table.py` creates `sdp_meta.pipeline_config`, and `transformations/01_dynamic_pipeline.py` builds bronze and silver tables from it.

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

SOURCE_PATH = spark.conf.get("source_path")
rows = [r.asDict() for r in spark.read.table(spark.conf.get("config_table"))
        .where("active = true").collect()]

def make_bronze(cfg):
    @dp.table(name=f"bronze_{cfg['entity']}_dyn")
    def _bronze(cfg=cfg):
        # default argument pins this row's values
        return (spark.readStream.format("cloudFiles")
                .option("cloudFiles.format", cfg["source_format"])
                .load(f"{SOURCE_PATH}/{cfg['source_subdir']}")
                .withColumn("_ingest_ts", F.col("_metadata.file_modification_time")))

    for cfg in rows:
        make_bronze(cfg)
```

Design point	Recommendation
Configuration	A Delta table or YAML file in Git, with columns for entity, path, format, keys, load type and expectations as JSON
Closures in loops	Wrap each dataset in a factory function, pass the row as a default argument, and set an explicit name=
Changes	The configuration is read when an update starts; a new row appears in the next update
Quality rules	Store expectations in the configuration and apply them with <code>dp.expect_all_or_drop</code>
When not to use it	Fewer than about ten sources, or sources that each need very different logic; plain code is easier to read

PART 4

Lakeflow Connect beyond SQL Server

The course demonstrated the managed SQL Server connector (see the *Lakeflow Connect* practical guide). Lakeflow Connect covers several other source types.

Connector type	Examples	Components
Database (CDC)	SQL Server, MySQL, PostgreSQL, Oracle	Connection, ingestion gateway, staging volume, ingestion pipeline
SaaS applications	Salesforce, Workday, ServiceNow, Dynamics 365, HubSpot, Jira	Connection and ingestion pipeline
Enterprise file services	SharePoint, Google Drive	Connection and ingestion pipeline; binary or parsed files
Query-based	Databases where CDC cannot be enabled	Periodic queries on a cursor column
Streaming	Kafka, RabbitMQ and other message buses	Continuous ingestion
Direct write	Zerobus Ingest API	Applications write records straight to streaming tables

Connectors are in different release stages, so check the connector list in the documentation for your cloud before planning a project. All managed connectors can be defined in bundles. The skeleton below shows the two pipelines of a database connector:

```
resources:
  pipelines:
    sqlserver_gateway:
      name: lfc-sqlserver-gateway-${bundle.target}
      continuous: true # the gateway must run continuously
      catalog: ${var.staging_catalog}
      schema: ${var.staging_schema}
      gateway_definition:
        connection_name: retailhub_sqlserver
        gateway_storage_catalog: ${var.staging_catalog}
        gateway_storage_schema: ${var.staging_schema}
    sqlserver_ingestion:
      name: lfc-sqlserver-ingestion-${bundle.target}
      serverless: true
      catalog: ${var.dest_catalog}
      schema: ${var.dest_schema}
      ingestion_definition:
        ingestion_gateway_id: ${resources.pipelines.sqlserver_gateway.id}
      objects:
        - schema:
            source_catalog: AdventureWorksLT
            source_schema: SalesLT
            destination_catalog: ${var.dest_catalog}
            destination_schema: ${var.dest_schema}
```

A job with a `pipeline_task` and a schedule then refreshes the ingestion pipeline. If you only need to query a database occasionally without copying data, use Lakehouse Federation instead of ingestion.

PART 5

System tables and observability

System tables in the system catalog record usage, jobs, access and lineage for the whole account. An account admin must grant access, for example `USE CATALOG on system` and `SELECT` on the schemas you need.

Table	Answers
<code>system.billing.usage</code> , <code>system.billing.list_prices</code>	DBU consumption per workspace, job, pipeline, warehouse or tag, and its list price
<code>system.lakeflow.jobs</code> , <code>job_run_timeline</code> , <code>job_task_run_timeline</code>	Job inventory, run outcomes and duration trends
<code>system.lakeflow.pipelines</code>	Pipeline inventory
<code>system.access.audit</code>	Who ran, changed or granted what, and when
<code>system.access.table_lineage</code> , <code>column_lineage</code>	Which jobs, notebooks and queries read and write each table
<code>system.compute.clusters</code> , <code>warehouses</code>	Compute configuration history
<code>system.storage.predictive_optimization_operations_history</code>	Automatic <code>OPTIMIZE</code> , <code>VACUUM</code> and <code>ANALYZE</code> operations

```
-- DBUs per job over the last 30 days
SELECT usage_metadata.job_id, sku_name, SUM(usage_quantity) AS dbus
FROM system.billing.usage
WHERE usage_metadata.job_id IS NOT NULL
      AND usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY ALL
ORDER BY dbus DESC;

-- Recent permission changes
SELECT event_time, user_identity.email, action_name, request_params
FROM system.access.audit
WHERE service_name = 'unityCatalog' AND action_name LIKE '%Permissions%'
ORDER BY event_time DESC
LIMIT 50;
```

Save queries like these in the SQL editor, add an alert (for example, when yesterday's DBUs exceed a threshold), and combine them into an AI/BI dashboard for the team. Custom tags on jobs, pipelines and warehouses appear in `system.billing.usage` and make cost allocation by project straightforward.

PART 6

OpenSharing (formerly Delta Sharing)

OpenSharing shares tables, views, volumes, models and notebooks with other organisations without copying data.

Databricks-to-Databricks sharing connects two Unity Catalog metastores through a sharing identifier. **Open sharing** serves any client, such as Spark, pandas or Power BI, using a bearer token or OIDC federation.

```
CREATE SHARE retail_partner_share COMMENT 'Daily revenue for the partner';
ALTER SHARE retail_partner_share ADD TABLE main.gold.daily_revenue;

-- Databricks-to-Databricks: the recipient sends you its sharing identifier
CREATE RECIPIENT partner_co USING ID 'azure:westeurope:<metastore-uuid>';
GRANT SELECT ON SHARE retail_partner_share TO RECIPIENT partner_co;

SHOW GRANTS ON SHARE retail_partner_share;
```

The recipient creates a catalog from the share and queries it like any other catalog. Sharing within one Databricks account is always enabled; sharing with other accounts or non-Databricks clients must be enabled on the metastore. Creating recipients requires **CREATE RECIPIENT** on the metastore. OpenSharing is also the basis of Databricks Marketplace and Clean Rooms.

PART 7

Lineage

Unity Catalog captures lineage automatically for queries run on Unity Catalog compute: which tables, columns, notebooks, jobs, pipelines and dashboards read or write each object. You need no extra code.

- **Catalog Explorer:** open a table, select **Lineage**, then **See lineage graph**. Select a column to follow its upstream columns.
- **System tables:** `system.access.table_lineage` and `system.access.column_lineage` keep a rolling one-year window for impact analysis in SQL.

```
-- Everything that wrote to or read from the silver orders table in the last week
SELECT entity_type, entity_id, source_table_full_name, target_table_full_name, event_time
FROM system.access.table_lineage
WHERE (source_table_full_name = 'main.silver.orders' OR target_table_full_name =
'main.silver.orders')
    AND event_date >= current_date() - INTERVAL 7 DAYS
ORDER BY event_time DESC;
```

Use lineage before you change or drop a table, to see which downstream jobs and dashboards will be affected. Tables referenced only by path are recorded by path, not by name. For systems outside Databricks, external lineage lets you add upstream and downstream objects manually.

PART 8

AI/BI dashboards and Genie

The bonus notebook `BONUS_aibi_dashboards` walks through both tools on RetailHub gold tables.

Tool	What it does	Tips
AI/BI dashboards	Datasets written in SQL, visualisations, filters and parameters; published with embedded credentials or viewer credentials; scheduled refresh and email subscriptions	Build on gold tables or materialized views; refresh from a job with a Dashboard task after the pipeline finishes
Genie spaces	Business users ask questions in natural language; Genie generates SQL against selected tables	Start with a few well-documented tables; add column comments, sample questions, instructions and trusted SQL queries; review the questions users actually ask

Both run on SQL warehouses and respect Unity Catalog permissions, row filters and column masks. A user sees only data they are allowed to query, unless a dashboard is published with the author's embedded credentials.

PART 9

Terraform for the environment

The course environment is described as code in `infra/terraform`. Treat it as a starting point for your own sandbox or team environment.

File	Creates
<code>main.tf, variables.tf</code>	Naming with a random suffix, tags (including <code>auto-delete-after</code>), resource group
<code>storage.tf</code>	ADLS Gen2 account with <code>raw</code> and <code>external</code> containers, Access Connector, Storage Blob Data Contributor role
<code>databricks_workspace.tf</code>	Premium workspace (optional; new workspaces start with a 14-day DBU trial)
<code>databricks_uc.tf</code>	Storage credential, external location and grants (when <code>configure_uc = true</code>)
<code>sql_server.tf</code>	Azure SQL server, serverless AdventureWorksLT database and firewall rules for Lakeflow Connect

```
cd infra/terraform
cp terraform.tfvars.example terraform.tfvars    # set prefix, passwords, IP addresses, dates
terraform init
terraform plan
terraform apply
terraform destroy                                # after the training; removes everything
```

```
resource "databricks_external_location" "external" {
  provider      = databricks.workspace
  name          = "el-${var.prefix}-training-external"
  url           = local.external_location_url
  credential_name = databricks_storage_credential.external[0].name
}
```

Next steps with IaC: manage catalogs, schemas, groups and grants with the Databricks provider (`databricks_catalog`, `databricks_schema`, `databricks_grants`); keep state in a remote backend such as an Azure Storage container with locking; and run `plan` in CI on pull requests. As a rule of thumb, Terraform manages the platform (workspaces, storage, Unity Catalog objects, identities) and bundles manage the workloads (jobs, pipelines, dashboards).

PART 10

CI/CD with GitHub Actions and bundles

Module 09 deployed `materials/cicd` from a terminal. In a team, the same commands run in CI whenever a pull request is opened or merged.

```
# .github/workflows/deploy.yml
name: Deploy RetailHub bundle
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      DATABRICKS_HOST: ${vars.DATABRICKS_HOST}
      DATABRICKS_CLIENT_ID: ${secrets.SP_CLIENT_ID} # service principal, OAuth M2M
      DATABRICKS_CLIENT_SECRET: ${secrets.SP_CLIENT_SECRET}
    defaults:
      run:
        working-directory: materials/cicd
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle validate -t test
      - if: github.event_name == 'pull_request'
        run: databricks bundle deploy -t test
      - if: github.event_name == 'push'
        run: |
          databricks bundle validate -t prod
          databricks bundle deploy -t prod
```

Practice	Why
Deploy as a service principal, never with a personal token	Deployments do not break when people leave, and permissions stay minimal
Prefer workload identity federation (GitHub OIDC) over client secrets	No long-lived secret is stored in the repository
Add a test target and run integration tests after deployment, for example <code>databricks bundle run -t test retailhub_job</code>	Errors are caught before production
Protect the production environment in GitHub with required reviewers	A person approves every production deployment
Keep <code>mode: production</code> and <code>run_as</code> in the prod target	Production resources are not renamed or paused and run under a stable identity

The prod target in `materials/cicd/databricks.yml` is a commented stub. Uncomment it and set its host, root path, variables and service principal before you use this workflow.

Resources

Resource	Address
Documentation (Azure)	learn.microsoft.com/azure/databricks
Documentation (all clouds)	docs.databricks.com
Certification	databricks.com/learn/certification
Exam registration	webassessor.com/databricks
Databricks Academy	databricks.com/learn/training
Free Edition	databricks.com/learn/free-edition
Community forums	community.databricks.com
Terraform provider	registry.terraform.io/providers/databricks/databricks
Bundle templates and CLI	github.com/databricks/cli